Date:

Toc H Institute of Science & Technology

(An ISO 9001:2015 Certified Institution)

Arakkunnam

INTERNAL ASSESSMENT MAIN ANSWER BOOKLET

	I	II	III	IV	V	VI	VII	VIII	IX	X	XI	XII	XIII
a	3	3	3	2	3	10	7	6					
b						4	7						
c													
d													
e													
f													
g													
h													
Total													
Grand Total	3	3	3	2	3	14	14	6					

TOTAL 48

V.good

TIST

Name	ARJUN A		
Branch & Semester	CSE-A S5		
Subject with Code	CST307 Microprocessors & micro controller		
Roll No.	32	University Reg. No:	T O C 2 3 C S 0 3 2
Signature of Invigilator	25/8/25	No. of Additional Sheets	1

1. 8085 microprocessor

→ 8085 is a 8 bit general purpose microprocessor.

→ It has 16 bit address bus.

→ It has 8 bit data bus

→ 8085 has 5 main ~~8b~~ registers

→ It is an assembly coded microprocessor.

→ It has 40 pins.

3

→ It works on +5v power supply.

→ It has a static frequency of 3mHz.

2. Pipeline Structure

8086 has a pipeline architecture, it makes 8086 faster and it has a instruction prefetch queue of 6 byte which help in pipelining.

Divided into :

3

Bus Interface Unit (BIU) : Fetching instruction from memory & I/O.

Execution Unit (EU) : For decoding and executing instruction.

When EU executes an instruction BIU fetches the next instruction. (pipelining (parallel working))

3. Signals used in maximum mode

① HOLD → When bus is requested by devices.
HLDA → when system gives the bus.

3

② RESET → It resets all registers to FFFF0H.

③ BHE → Bus high enable.

④ INTA → Interupt Acknowledge → used after interupt req is being satisfied.

4. Effective address = Segment x10H + offset.

(i) MOV [AX], [BX]

EA = S x10H + offset

= ~~10000~~ (~~AX~~ + BX) x10H + 5000H

= 2000H x10H + 5000

= 25000H

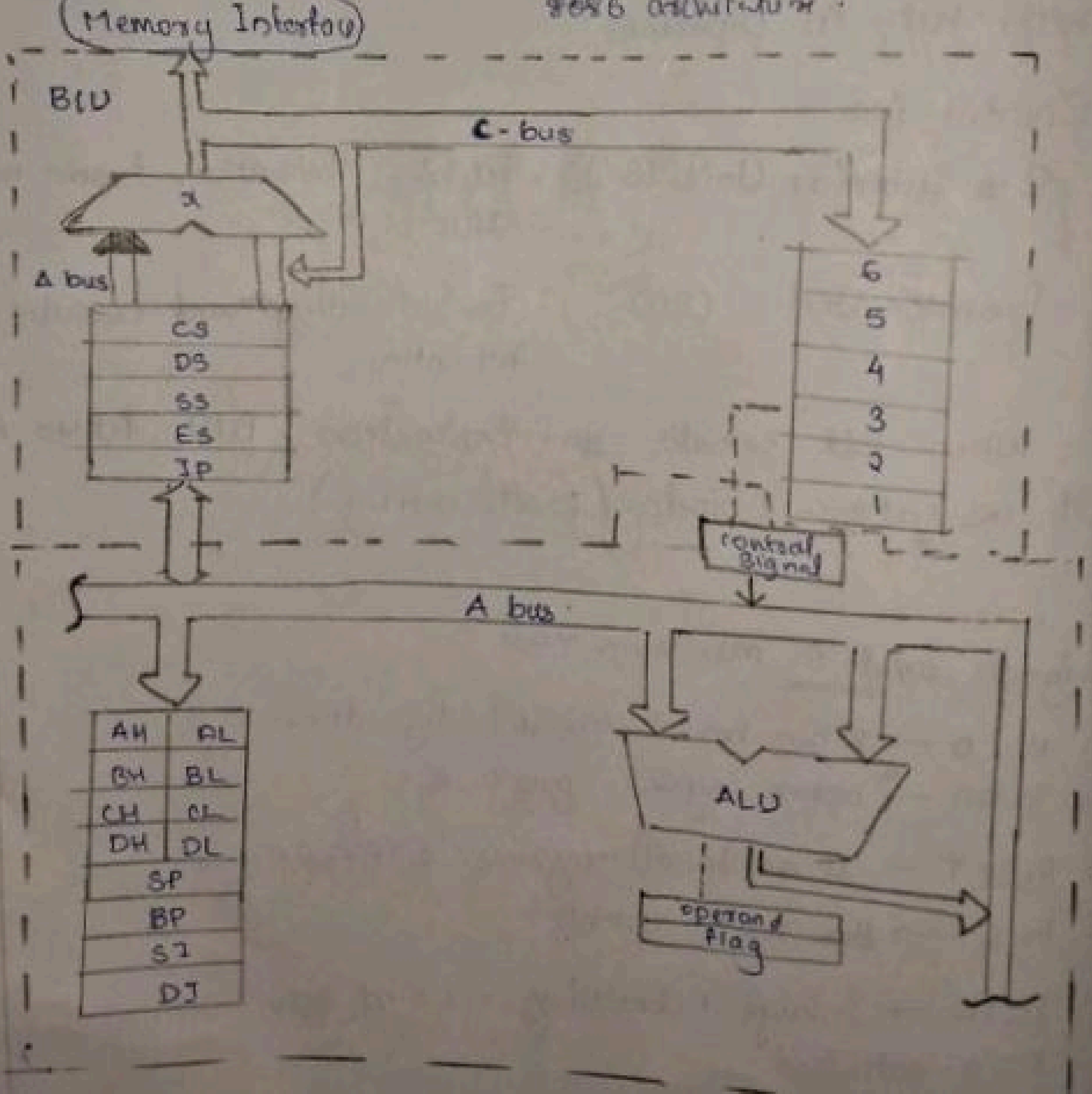
(ii) MOV AX, 5000[BX].

EA = (5000 + 2000)H x10H + 5000

= 7000 x10 + 5000

= 75000H

6(a) 8086 architecture.



8086 is a 16 bit ~~[illegible]~~ microprocessor having 20bit address bus and 16bit data bus. Has 14 registers associated with it and works on +5v power supply.

It includes:- It is divided into 2 part : BIU & EU.
BIU : Fetch instruction from memory / I/O it includes:-

1) Instruction Prefetch Queue

→ It is a 6byte FIFO structure.
→ It is used to store pre-fetched data instructio from memory.
→ It help is faster executio.

2) Segment Register.

→ It has segment register for easy segmentatio.
→ It include-
CS (code segment) - Points to instructio
DS (Data segment) - Point to data
SS (Stack ") - Point to top element of stack.
ES (Extra ") - Used for string operation.

EU :→ It stands for execution unit.

→ It is responsible for decoding and executing instructi.

It includs

1) ALU (Arithmetic & Logic Unit)

→ It is used for performing arithmetic & logical calculation

eg: Arithmetic calc such as : ADD, SUB, MUL
Logical " : AND NOT NAND.

2) It includes General Purpose

1) AX → Accumulator register.
→ used for storing ALU calculation values.
→ split as AH & AL

2) BX → Base register.
→ used for storing base address of some registers.
→ split as BH & BH

3) CX → Counting Register.
→ used for counting
→ used with LOOP, REP function for tacking the count.
→ split as CH & CL

4) DX — Data register.
→ used for storing valid data.
→ split as DH & DL

5) Stack pointer : Used for pointing to top of stack.
6) Base : Used for pointing to base register.
7) Source Index : Used for pointing to the source address
8) Destination Index : Used for pointing to the destination ad

3) Flag

It contains flags-

Carry Flag

Zero Flag

Parity Flag

Auxiliary carry.

Sign Flag

6b

8086 Microprocessor	8088 microprocessor.
→ It is a 16 bit microprocessor	→ It is also a 16 bit micro processor.
→ It has 20 bit address bus	→ It has 20 bit address bus.
→ It has 16 bit data bus	→ It has 8 bit data bus
→ It has a 6 byte pre-fetch queue	→ It has 4 byte pre-fetch queue.
→ It fetches 2 byte instructions at a time.	→ It fetches 1 byte at a time.
→ works in 3 MHz	→ Has variable frequency. 5-10 MHz.
→ It is faster.	→ It is slower.
→ It is assembly based	→ It is general purpose reg based.

4

7(a)(a) CMP (compare) - It is used to compare 2 instruction

- If result is 0, then its not same else its same.

eg: CMP AX, BX

(b) MUL (multiply) - It is an instruction which require only 1 reg inputs.

- It multiplies the given value with the value to accumulator and store it in accumulator.

eg: MUL #4 (multiply 4 to AC)

(c) DIV (divide) - It is also an instruction which require only 1 reg input.

- It divides the given value accumulator value using in value given in instruction.

(d) RCR - Rotate Carry Right
- It is used to rotate the carry generated to the right.

- eg: RCR AX, BX

(e) ROR - Rotate Right
- It is used to rotate the value of a register to right.

- eg: ROR AX

7b. Addressing mode of 8086

The addressing modes of 8086 is divided into 2:
1) Addressing mode for ~~instruction~~ data
2) Addressing mode for ~~data~~ instruction

1) Addressing mode for ~~register~~ ~~instruction~~ data

(1) Immediate Addressing mode.
→ The operand is directly given in the ~~reg~~ instruction.

eg: MOV AX, 2000H.

(2) Register Addressing mode
→ The operand is stored inside a register
→ MOV AX, BX.

(3) Direct addressing mode
→ The memory location where the operand is present is given in the instruction
eg :- MOV AX, [5000H]

(4) Indi...

→ The EA is present inside a regstr which point towards a memory location where the operand is present.

→ eg:- MOV AX, [BX]

// [BX] contain a mem loc which has the operand.

(5) Indexed Addrng mode

→ It is done using index regstr.

→ EA = Index reg + displacemnt.

(6) Base Addresing mode

→ It is done using base regstr.

→ EA = Base reg + disp.

(7) Base-Index Addrng mode.

→ It is done using both base & index

→ EA = Base + Index.

(8) Base Index with ~~displt~~ displacemnt

→ EA = Base + Indx + Disp.

② Addr mode for ~~data~~ instructn.

→ Direct Addrng mode.

eg: JMP 3000H.

It operand is directly provided.

→ Indirect Addresng mode.

eg JMP [3000H].

5. ~~Regis~~ Data ~~Tra~~ Transfer Instructi.

- It is used for the transmission of data b/w registers.

① ~~Mov~~ MOV dest, source - Used to move / copy data b/w reg.
(reg)/(mem)

eg: MOV AX, BX (BX)

② PUSH (Stack) - It is used to insert an elmt to stack.
- Stack pointer (↓) ~~~~ decrements.

eg: PUSH 20004

③ ~~~~ XCHG (reg) - It is used to swap b/w 2 regs.

④ POP (Stack) - It is used to delete an elmt from stack.
- SP (↑)
- POP 20004

⑤ IN/R0 - Used for I/O operat.
- This specifically used for reading input from R0.

⑥ OUT/R1 - Used for I/O operat.
- This specifically used for writing to R1.

3

8.

```
1000        CLC
1000        MOV SI, 27000
            MOV DI, 3000
            MOV CH, 00 // set CH as the total no's.
    LOOP    MOV AX, [SI]
            INC SI
            INC SI.
            INC CX
            JLT CX, CH (Checking if CX < CH) : LOOP
                 If yes get next no from user.
COM         AX, CH.
```

1000 CLC

1000 MOV SI, 2700

MOV DI, 3000

MOV CH, 00

LOOP MOV AX, (SI)

INC SI

INC SI.

INC CX

JLT CX, CH.

COM AX, CH.

JNZ L1:

INC CH

L1 : MOV (DI), AX

INC DI

INC DI.

MOV DI, (CH).

L1.

MOV DI, [AX]

JNZ

Give sample i/p & o/p

Ak 21/8

Now we have the smallest no in DI / DH or DL

Thus found out the smallest no in array entered by us.